

University of Rochester
Warner School of Education

Innovative Unit Plan: Minecraft Education Coding
FUNdamentals

Inclusive Unit Planning for Neurodivergent & CLD Learners
Minecraft Education as an Equity-Centered CS Context

Piter Garcia
Teaching & Curriculum Graduate Student

March 19, 2026

ED452B — Instructional Strategies for Inclusive Classrooms B

Design Statement: This first-pass innovative unit plan adapts the Minecraft Education: Coding FUNdamentals sequence into an inclusive elementary computer science unit grounded in MTSS, UDL, and culturally and linguistically sustaining design. The central planning commitment is to avoid conflating access barriers with competence, especially for neurodivergent and culturally and linguistically diverse learners.

Contents

1	Introduction	2
2	Unit Context and Rationale	2
2.1	Unit Context	2
2.2	Community, School, and Classroom Characteristics	2
2.3	Broader Curricular Context	2
2.4	Rationale for Unit Design	3
3	Theoretical Framework	3
4	Objectives	4
4.1	Unit Goal	4
4.2	Essential Question	4
4.3	Big Ideas	4
4.4	Unit Objectives	4
4.5	Relevant Standards	4
5	Assessment	5
5.1	Assessment Plan	5
5.2	Assessment Principles	5
6	Pedagogy	6
6.1	Instructional Approach	6
6.2	Core Teaching Strategies	6
6.3	Supports for Diverse Learners	6
7	Unit Implementation	7
7.1	Four-Lesson Sequence	7
7.2	Lesson 1: Basic Moves	7
7.3	Lesson 2: Sequencing to Reach a Goal	8
7.4	Lesson 3: Debugging and Revision	8
7.5	Lesson 4: Efficient Solutions and Reflection	9
8	Target Students and Differentiation	9
9	Analysis of Student Learning	9
10	Unit Analysis	10

1 Introduction

This innovative unit plan is built around *Minecraft Education: Coding FUNdamentals*, specifically the *Animal Research Center* sequence. The unit is designed for an inclusive Grade 4–5 IGNITE computer science rotation at Pine Brook and spans four 45-minute lessons. The purpose of the unit is to teach foundational computational thinking through a high-interest digital environment while making the learning sequence accessible to neurodivergent and culturally and linguistically diverse learners.

The unit is innovative in two ways. First, it uses a game-based coding environment that allows students to learn algorithms, sequencing, debugging, and problem solving through immediate visual feedback. Second, it redesigns that environment through inclusive instructional structures so that access to the lesson is not limited by reading load, startup difficulty, executive-function demands, or prior familiarity with Minecraft.

2 Unit Context and Rationale

2.1 Unit Context

Candidate:	Piter Garcia
School / Setting:	Pine Brook, IGNITE elementary computer science rotation
Grade Band:	Grade 4–5
Content Area:	Computer Science / Digital Literacy
Unit Title:	Minecraft Education Coding FUNdamentals — Animal Research Center
Unit Length:	4 lessons, approximately 45 minutes each
Instructional Setting:	rotating inclusive classes with mixed readiness, mixed digital fluency, and varied support needs

2.2 Community, School, and Classroom Characteristics

Pine Brook’s IGNITE program serves elementary learners through short rotation-based computer science instruction. Students enter the classroom with uneven prior experiences in coding, digital navigation, keyboard use, reading stamina, and collaborative work habits. The placement transcripts further show that students vary in how independently they can follow multi-step digital directions, persist through troubleshooting, and ask for help appropriately. The classroom context also includes neurodivergent learners and culturally and linguistically diverse learners whose competence may not be visible when a lesson depends heavily on fast reading, long verbal directions, or self-managed transitions.

2.3 Broader Curricular Context

This unit sits at the beginning of a block-coding progression in the IGNITE rotation. It introduces the Agent, MakeCode, step-by-step algorithms, and basic debugging routines that can later transfer to other coding environments already used in placement, including Code.org and robotics activities. The unit therefore functions as an entry point into programming rather than as an isolated enrichment activity.

2.4 Rationale for Unit Design

This unit is appropriate for the setting because it teaches core computer science practices through a motivating environment while still allowing direct observation of student thinking. Placement evidence indicates that students often need support with startup, reading NPC directions, understanding the task goal, and sustaining effort when code fails. For that reason, the unit is designed to separate *coding understanding* from *interface difficulty*. The lesson sequence keeps the official Minecraft tasks, but it adds supports that make the learning goals more reachable and the evidence of learning more valid.

Placement-Informed Design Decisions

- › whole-class launch routine for the click-path into the lesson
- › explicit routine for reading NPC directions before coding
- › structured peer support through driver / navigator roles
- › planning and debugging supports before students reach longer code strings
- › assessment that distinguishes startup problems from conceptual coding problems

3 Theoretical Framework

This unit is grounded in an inclusive design framework that treats support as part of the lesson rather than as a remedial add-on. Karten’s work on inclusive instruction supports the view that classroom design should anticipate learner variability and provide access to rigorous tasks through proactive scaffolding. In this unit, that principle appears in the use of visual supports, predictable routines, explicit modeling, and layered assistance during coding.

The unit is also informed by Universal Design for Learning. Students are provided multiple means of engagement, representation, and expression. For example, the learning sequence includes teacher modeling, visual reminders, oral explanation, partner discussion, planning sheets, and more than one way to demonstrate understanding. These features are especially important in computer science, where students may understand the logic of a task but struggle to show it because of reading load, interface management, or the pace of the lesson.

MTSS further shapes the unit by clarifying the relationship between universal supports and targeted supports. Tier 1 structures such as click-path charts, predictable routines, and a common stuck protocol are designed for the whole class. Tier 2 supports such as buddy reading of NPC directions, shortened paths, pre-highlighted controls, and teacher check-ins are layered in for students who need additional access. Together, these frameworks justify an innovative unit that preserves rigor while increasing access.

4 Objectives

4.1 Unit Goal

Students will develop foundational computational thinking by using block coding in Minecraft Education to create, test, debug, and explain algorithms.

4.2 Essential Question

How can students use algorithms, debugging, and collaboration to solve coding problems in a digital environment?

4.3 Big Ideas

- › Algorithms are precise step-by-step instructions.
- › Debugging is an expected part of coding.
- › Planning, testing, and revising improve program quality.
- › Inclusive supports help more learners participate meaningfully in computer science.

4.4 Unit Objectives

- › Students will create and test algorithms using Minecraft Education's block-coding tools.
- › Students will use sequencing, debugging, and planning to improve code.
- › Students will explain how their code works using computer science vocabulary.
- › Students will collaborate productively through structured peer roles.

4.5 Relevant Standards

- › **CSTA 1A-AP-08:** Model daily processes by creating and following algorithms.
- › **CSTA 1A-AP-10:** Develop programs with sequences and simple loops.
- › **CSTA 1A-AP-11:** Decompose steps needed to solve a problem into a precise sequence of instructions.
- › **CSTA 1A-AP-14:** Debug errors in an algorithm or program that includes sequences and simple loops.
- › **CSTA 1B-AP-08:** Compare and refine multiple algorithms for the same task.

5 Assessment

5.1 Assessment Plan

Lesson	Focus	Lesson Goal	Formative Assessment	Evidence Collected
Lesson 1	Basic Moves	Students will navigate Minecraft Education, access MakeCode, and use a short sequence of movement blocks.	teacher observation of startup and navigation; prediction before Run ; quick conference during first code attempt; exit ticket	startup independence, sequence accuracy, basic error noticing
Lesson 2	Sequencing	Students will plan and code a longer algorithm to reach a goal.	planning sheet review; observation of partner talk; teacher check-in before code execution	planning quality, sequence accuracy, use of vocabulary
Lesson 3	Debugging	Students will identify errors, revise code, and explain how a fix improved the result.	debug conference; code revision check; peer explanation	evidence of revision, explanation of bug and fix
Lesson 4	Efficient Solutions	Students will compare solution paths, complete a culminating challenge, and reflect on their learning.	completion conference; reflection response; rubric-based review	efficiency of solution, explanation, final reflection

5.2 Assessment Principles

How Learning Will Be Evaluated

- › Students are not assessed only by whether they finish the Minecraft task.
- › Assessment includes planning, explanation, debugging, collaboration, and task completion.
- › Teacher notes distinguish between procedural difficulty, reading difficulty, and conceptual difficulty.
- › Exit tickets and conferences provide evidence for learners who do not demonstrate understanding fully through the in-game task alone.

6 Pedagogy

6.1 Instructional Approach

The unit uses explicit instruction, guided practice, structured collaboration, and reflective closure. Each lesson follows a stable sequence: warm-up, teacher model, guided practice, independent or partner work, and reflection. This structure supports pacing and predictability, which are especially important in a short rotation model.

6.2 Core Teaching Strategies

- › think-aloud modeling during startup, navigation, and coding
- › explicit teaching of vocabulary and controls
- › partner roles to structure collaboration
- › planning before execution
- › immediate feedback through testing and debugging
- › short reflective writing or oral explanation at lesson end

6.3 Supports for Diverse Learners

Area of Need	Instructional Response
Startup and navigation difficulty	click-path chart; whole-class launch routine; teacher modeling of lesson entry
Reading load / NPC directions	partner read-aloud; teacher restatement; visual reminders to read before coding
Executive-functioning needs	chunked directions; visible checklist; planning sheet before longer sequences
Social / regulation needs	predictable partner roles; clear stuck protocol; shorter check-ins
Language-processing needs	simplified directions; oral rehearsal; vocabulary support; sentence frames

7 Unit Implementation

7.1 Four-Lesson Sequence

Lesson	Title	Main Content	Key Activities	Products
1	Basic Moves	movement controls, Agent, MakeCode entry	launch routine, teacher model, short movement code, exit ticket	short sequence and reflection
2	Sequencing to Reach a Goal	longer ordered algorithms	plan-before-you-code sheet, guided sequence build, partner coding	planning sheet and working code
3	Debugging and Revision	identifying and fixing errors	debug mini-lesson, peer explanation, revision cycle	revised code and bug explanation
4	Efficient Solutions and Reflection	comparing solutions, final challenge	challenge task, reflection, teacher conference	final solution and unit reflection

7.2 Lesson 1: Basic Moves

Lesson Goal:	Students will navigate Minecraft Education, open MakeCode, and use basic movement blocks to move the Agent toward a goal.
Objectives:	Students will use movement controls, access MakeCode, write a short sequence, and test whether the Agent does what they expect.
Materials:	Minecraft Education, Chromebook or device, click-path chart, control reminder, exit ticket.
Opening:	Unplugged directional warm-up and whole-class launch routine into the lesson.
Direct Instruction:	Teacher models the lesson path, reading the NPC directions, entering the yellow-walled room, opening MakeCode, and building the first short sequence.
Guided Practice:	Students code along with the teacher and predict the Agent's movement before running the code.
Independent / Partner Work:	Students complete the first movement task with peer support.
Closure:	Students complete an exit ticket describing what their code did and one thing they had to fix.
Assessment:	Teacher observation, prediction routine, and exit ticket.

7.3 Lesson 2: Sequencing to Reach a Goal

Lesson Goal:	Students will plan and write a longer sequence to move the Agent to a goal.
Objectives:	Students will map a path, code an ordered sequence, and explain their reasoning to a partner.
Materials:	Minecraft Education, planning sheet, vocabulary support, teacher checkpoint notes.
Opening:	Warm-up comparison of two short algorithms.
Direct Instruction:	Teacher models how to use a planning sheet before building code.
Guided Practice:	Class co-constructs a path and matching code.
Independent / Partner Work:	Students solve a new challenge using the planning sheet when needed.
Closure:	Students explain their algorithm to a partner and revise one step if necessary.
Assessment:	Planning sheet, partner explanation, and teacher observation.

7.4 Lesson 3: Debugging and Revision

Lesson Goal:	Students will identify coding errors and revise their algorithm.
Objectives:	Students will compare expected and actual outcomes, identify an error, and revise the program.
Materials:	Minecraft Education, debug prompt sheet, reflection prompt.
Opening:	Short teacher demonstration of a broken program.
Direct Instruction:	Teacher models a simple debugging process: predict, test, identify the bug, revise, retest.
Guided Practice:	Class corrects one shared code error together.
Independent / Partner Work:	Students revise their own code and explain what changed.
Closure:	Students submit a short explanation of the bug and fix.
Assessment:	Debug conference and revision evidence.

7.5 Lesson 4: Efficient Solutions and Reflection

Lesson Goal:	Students will compare solution strategies and complete a culminating coding challenge.
Objectives:	Students will complete a final task, compare two possible algorithms, and reflect on their coding growth.
Materials:	Minecraft Education, final reflection, simple rubric.
Opening:	Review of sequencing and debugging strategies.
Direct Instruction:	Teacher models how a solution can be correct but more or less efficient.
Guided Practice:	Students compare two sample solutions.
Independent / Partner Work:	Students complete the culminating task and conference with the teacher.
Closure:	Students reflect on what helped them succeed in the unit.
Assessment:	final code, explanation, and reflection.

8 Target Students and Differentiation

Target Student	Likely Barrier	Planned Support
Neurodivergent learner with executive-functioning needs	skipping steps, acting before reading directions, frustration when the code does not work immediately	visible checklist, chunked directions, prediction before <code>Run</code> , planning sheet, teacher checkpoint
CLD learner with language-processing / reading-load needs	dense NPC text, unfamiliar vocabulary, fast oral directions	partner read-aloud, simplified directions, visual vocabulary support, oral explanation options
Neurodivergent learner with regulation / social-communication needs	overload during noisy partner work or repeated resets	explicit driver / navigator roles, predictable routine, calm stuck protocol, brief teacher check-ins

9 Analysis of Student Learning

Student learning in this unit will be analyzed through patterns across the four lessons rather than through completion alone. The key questions are whether students increasingly demonstrate independence with algorithms, whether they can explain how their code works, and whether debugging becomes more strategic over time. In addition, the teacher will examine whether supports such as planning sheets, partner roles, and click-path charts improve access without lowering expectations.

10 Unit Analysis

The strength of this unit is that it preserves the authentic Minecraft coding sequence while organizing it into a more explicit instructional design. Students engage in real coding tasks, but the unit does not assume that motivation alone will produce access. Instead, it pairs a high-interest platform with clear routines, support structures, and layered assessment. For an inclusive classroom, that combination is more instructionally responsible than assigning the digital task without redesign.

The next revision of the unit should be informed by student work, teacher notes, and observed patterns of support. That second version can then be tightened to the rubric with concrete evidence of what actually helped learners succeed.

References

- [1] Karten, T. J. *Inclusion Strategies and Interventions*. Warner course text.
- [2] CAST. *Universal Design for Learning Guidelines*. Retrieved from <https://udlguidelines.cast.org>
- [3] Minecraft Education. *Coding FUNDamentals: Animal Research Center*. Curriculum materials used as the basis for this unit plan.
- [4] Teaching placement transcripts, March 2026. Local classroom evidence used to refine implementation decisions for Pine Brook IGNITE.